

Contents

Coffee <-> Stomach Upset Example	2
Goal	2
Step 0: The Structural Causal Model (SCM)	2
Step 1: Abduction (Infer the latent variable U)	2
Step 2: Modification (Set the counterfactual action $T := 0$)	3
Step 3: Prediction (Calculate the counterfactual outcome)	3
Summary Table	3
Key Takeaways	3
How and why was $Y = TU + (T-1)(U-1)$ constructed the way it was?	4
Goal of the Equation	4
Constructing the Equation	4
So what's going on?	6
Final Notes	6
$Y = 1$ if $U = 0$ and $T = 0$?	7
So what's going on?	7
The correct expression	7
So why use the weird version?	8
What to do?	8
The Tradeoff in SCMs	9
How Do Researchers Handle This?	9
1. Piecewise-defined structural equations	9
2. Latent variables + noise	9
3. Graph-based SCMs (do-calculus)	10
Summary	10
A More Complex Example - Heart Health	10
Scenario: Medication $\rightarrow$ Blood Pressure $\rightarrow$ Heart Health	11
Step 1: Install Required Libraries	11
Step 2: Define the SCM in Python	11
Step 3: Visualize the Causal Graph	12
Step 4: Abduction + Counterfactual	13
Step 5: Compute Counterfactual	13
Example Output	14
Want to go further?	14
4-Part Causal Inference Exploration Roadmap	14
Part 1: Learn an SCM from data	14
Part 2: Add confounding	14
Part 3: Apply do-calculus	14
Part 4: Generate probabilistic counterfactuals	14
Part 1: Learn SCM from Data (with DoWhy)	14
Part 2: Add a Confounder and Adjust	16
Part 3: Apply Do-Calculus with DoWhy	18
Part 4: Generate Probabilistic Counterfactuals via Monte Carlo	19

Mediation Analysis	21
Instrumental Variables	21
Time series SCMs	21

Coffee <-> Stomach Upset Example

Let's walk through the **coffee counterfactual example** step by step with explanations, so you can understand how it was constructed and how each part of the counterfactual framework (abduction, modification, prediction) applies to it.

Goal

We are computing a **counterfactual**:

*“What would have happened if you **had not** drunk coffee, even though in reality you **did** drink coffee and felt bad?”*

Step 0: The Structural Causal Model (SCM)

The SCM (Structural Causal Model) has two structural equations:

- **Treatment variable:** $T := t$ (We externally set T , like choosing to drink coffee or not.)
- **Outcome variable:**

$$Y := TU + (T - 1)(U - 1)$$

This equation defines how your **reaction** Y depends on your **coffee sensitivity** U and whether you drank coffee T .

- **Latent/exogenous variable** $U \in \{0, 1\}$:
 - $U = 1$: coffee-sensitive
 - $U = 0$: not coffee-sensitive

Step 1: Abduction (Infer the latent variable U)

We are told:

- $T = 1$ (you **drank** coffee)
- $Y = 1$ (you **felt bad**)
- But U is **unknown**

So we **solve** the equation for Y to find U .

Original equation:

$$Y = TU + (T - 1)(U - 1)$$

Substitute in the known values $T = 1, Y = 1$:

$$1 = 1 \cdot U + (1 - 1)(U - 1) = U + 0 \cdot (U - 1) = U$$

So,

$$U = 1$$

Abduction result: You are **coffee-sensitive**.

Step 2: Modification (Set the counterfactual action $T := 0$)

Now, we simulate an alternate world where you **didn't** drink coffee:

$$T := 0$$

Use the **same structural equation**, but with $T = 0$ and $U = 1$:

$$Y = TU + (T - 1)(U - 1)$$

Step 3: Prediction (Calculate the counterfactual outcome)

Substitute the values:

$$Y = 0 \cdot 1 + (0 - 1)(1 - 1) = 0 + (-1)(0) = 0$$

Counterfactual outcome: If you **hadn't drunk coffee**, you **would not** have felt bad.

Summary Table

Step	Input	Computation	Result
Abduction	$T = 1, Y = 1$	Solve $Y = TU + (T - 1)(U - 1)$	$U = 1$
Modification	Set counterfactual $T = 0$	Use same SCM	$T = 0$
Prediction	Use $U = 1, T = 0$	Compute $Y = TU + (T - 1)(U - 1)$	$Y = 0$

Key Takeaways

- **SCM equations must be known.**
- **Exogenous variable** U is unobserved but inferred from evidence — this is the core of *abduction*.
- **Counterfactuals are computed by modifying the treatment** and then using the same model to predict the outcome under this change.

How and why was $Y=TU+(T-1)(U-1)$ constructed the way it was?

Goal of the Equation

We want to define a causal model where:

- $T = 1$ means the person drinks coffee.
- $U = 1$ means the person is coffee-sensitive.
- Y is the **reaction**:
 - $Y = 1$ means the person feels bad.
 - $Y = 0$ means the person feels fine.

We want the model to obey the following *causal logic*:

Coffee-sensitive (U)	Drank coffee (T)	Felt bad (Y)
0	0	0
0	1	0
1	0	0
1	1	1

So the **only case** where you feel bad is:

You're coffee-sensitive **and** you drink coffee.

Constructing the Equation

We want to encode the **AND** logic of:

“ $Y = 1$ **if and only if** $T = 1$ and $U = 1$ ”

That is the behavior of **logical AND**. In math, the product of two binary variables captures this:

$$Y = T \cdot U$$

But then what's with this:

$$Y = TU + (T - 1)(U - 1)$$

Why not just $Y = TU$? Because the author wants a **fully general structural equation** that:

- Works for **binary inputs** $T, U \in \{0, 1\}$
- Produces binary outputs $Y \in \{0, 1\}$
- Is **differentiable** and **symmetric** in a sense
- Allows **algebraic manipulation** to solve for U , which is needed in **abduction**

So let's **analyze this form**:

$$Y = TU + (T - 1)(U - 1)$$

Let's expand the second term:

$$(T - 1)(U - 1) = TU - T - U + 1$$

So:

$$Y = TU + (TU - T - U + 1) = 2TU - T - U + 1$$

You can write this in closed form:

$$Y = 2TU - T - U + 1$$

So this function is:

- 1 when $T = 1$ and $U = 1$
- 0 otherwise

Let's verify all 4 binary cases:

T	U	$2TU - T - U + 1$	Interpretation
0	0	$0 - 0 - 0 + 1 = 1 - 0 - 0 = 1 - 0 - 0 = 1 \rightarrow$	should be 0
0	1	$0 - 0 - 1 + 1 = 0 \rightarrow$	
1	0	$0 - 1 - 0 + 1 = 0 \rightarrow$	
1	1	$2 - 1 - 1 + 1 = 1 \rightarrow$	

Wait — **this suggests it gives 1 when $T = 0$ and $U = 0$** , which violates our logic! So let's go back — this looks wrong.

Let's evaluate the original expression **without expansion**:

$$Y = TU + (T - 1)(U - 1)$$

Let's plug in all four possible (T, U) combinations:

Case 1: $T=0, U=0$

$$Y = 0 + (-1)(-1) = 0 + 1 = 1 \text{ (should be 0)}$$

Case 2: $T=0, U=1$

$$Y = 0 + (-1)(0) = 0$$

Case 3: $T=1, U=0$

$$Y = 0 + (0)(-1) = 0$$

Case 4: $T=1, U=1$

$$Y = 1 + 0 = 1$$

So clearly, this function gives $Y = 1$ for both $(1,1)$ and $(0,0)$ — which doesn't match the desired behavior. That seems **counterintuitive** if we just want the AND logic.

So what's going on?

Actually, the confusion arises because the **intended structure** is:

$$Y := \begin{cases} 1 & \text{if } T = 1 \text{ and } U = 1 \\ 0 & \text{otherwise} \end{cases}$$

Which is just $Y = T \cdot U$. But this more complicated form:

$$Y = TU + (T - 1)(U - 1)$$

is an **algebraic trick** to make Y a **smooth quadratic function** in T and U over $\{0, 1\}$, where you can **solve for** U directly.

Indeed, rearranging it for U :

$$Y = TU + (T - 1)(U - 1)$$

Solve for U :

Distribute second term:

$$Y = TU + (T - 1)(U - 1) = TU + (TU - T - U + 1) \Rightarrow Y = 2TU - T - U + 1$$

Then solving for U leads to:

$$u = \frac{Y - 1 + T}{2T - 1}$$

Which is only defined for $T \neq \frac{1}{2}$, i.e., $T \in \{0, 1\}$. So, in fact, the function is:

- Designed for **algebraic manipulability**
- Matches the desired counterfactual logic when $T = 1$
- And is **invertible** — you can solve for U

Final Notes

- **Why use** $Y = TU + (T - 1)(U - 1)$?

Because it satisfies:

- $Y = 1$ iff $T = 1, U = 1$
- Algebraically invertible to recover U
- Smooth polynomial in binary inputs

You can think of it as a clever encoding of conditional logic that supports **abduction** (solving for U) without piecewise definitions.

$Y = 1$ **if** $U = 0$ **and** $T = 0$?

If $U = 0$ and $T = 0$, then $Y = 0 * 0 + (0 - 1) * (0 - 1) = 0 + 1 = 1$... which means if i'm not coffee-sensitive and i didn't have coffee, i don't feel well? Why is the SCM set up like that?

If we take the structural equation as written:

$$Y = TU + (T - 1)(U - 1)$$

and plug in:

- $T = 0$ (you **didn't** drink coffee)
- $U = 0$ (you are **not** coffee-sensitive)

Then:

$$Y = 0 \cdot 0 + (0 - 1)(0 - 1) = 0 + 1 = 1$$

That implies:

You **didn't drink coffee**, you're **not sensitive**, and yet you **feel bad** — which is nonsense!

So what's going on?

This means the equation **does not faithfully encode the intended causal logic** in all cases. That equation:

$$Y = TU + (T - 1)(U - 1)$$

only **works correctly** when you're trying to model a very specific case where **you observe the outcome under treatment** and want to **invert the model for** U . It was crafted more for **abduction** in that specific setting than for **truthful semantic behavior in all scenarios**.

In other words:

- It gives the correct result for $T = 1, Y = 1$ to solve for $U = 1$
- But **logically fails** when $T = 0, U = 0$

The correct expression

If your intent is simply to express:

“You feel bad **if and only if** you drank coffee **and** are sensitive”

Then the **correct structural equation** should just be:

$$Y := T \cdot U$$

Let's verify all 4 combinations:

T	U	$Y = T \times U$
0	0	0
0	1	0
1	0	0
1	1	1

This matches the semantics you want.

So why use the weird version?

Because in counterfactual reasoning, you often want a **function that is invertible** — that you can **solve for latent variables like U** given observed Y and T . For example, if you just used $Y = T \cdot U$, you **can't solve for U** when $T = 0$, because:

$$Y = 0 \Rightarrow U \text{ could be anything!}$$

The more complex equation was crafted **so that when $T = 1$** , you can solve:

$$Y = TU + (T - 1)(U - 1) \Rightarrow U = \frac{Y - 1 + T}{2T - 1}$$

So the trade-off is:

Equation	Pros	Cons
$Y = T \cdot U$	Simple, logical	Not invertible for abduction
$Y = TU + (T - 1)(U - 1)$	Invertible for abduction	Logically wrong in some input cases

What to do?

If you're modeling a real system, use $Y = T \cdot U$

If you're walking through a **specific counterfactual reasoning example**, where you observe:

- $T = 1$
- $Y = 1$

and want to **back out U** , then the more complex expression helps **but only in that limited scope**.

This is one of the **core challenges in structural causal modeling**:

Designing structural equations that are both **semantically faithful** and **algebraically manipulable** becomes exponentially harder as your model complexity increases.

Let's unpack this and show you what's really going on under the hood, and how it's addressed in more serious causal work.

The Tradeoff in SCMs

When building SCMs, you're balancing:

Goal	Challenge
Semantic faithfulness	Does the equation reflect the real-world causal logic?
Algebraic invertibility	Can I solve for latent variables (like U) given observed data (like T, Y)?
Counterfactual support	Can I simulate "what if" scenarios easily from the model?

The **coffee model** tries to do all three but compromises semantics in the $T = 0, U = 0$ case.

In **realistic models**, these tradeoffs become harder:

- There are **many variables**
- The relationships can be **nonlinear**, **non-binary**, or **stochastic**
- There may be **latent confounding** that you can't write down directly

How Do Researchers Handle This?

1. Piecewise-defined structural equations

Instead of writing a single formula, define causal behavior by case:

```
def Y(T, U):  
    if T == 1 and U == 1:  
        return 1  
    else:  
        return 0
```

- Semantically faithful
- But, not algebraically invertible (harder to solve for U)

2. Latent variables + noise

Instead of hard logic, introduce a **noise term**:

$$Y := f(T, U, \varepsilon)$$

Where:

- $\varepsilon \sim \mathcal{N}(0, 1)$ or some known distribution
- f is a function capturing smooth causal influence

This allows you to do **probabilistic inference** and **Bayesian abduction**, even if f isn't invertible algebraically.

For example, in a model:

$$Y := \sigma(T \cdot U + \varepsilon)$$

where σ is a sigmoid

Now:

- You can infer U probabilistically via **posterior inference**
- You can compute counterfactual distributions

3. Graph-based SCMs (do-calculus)

In complex causal graphs, you define:

- **Nodes** for variables
- **Edges** for structural influence
- Functions f_i for each variable

Then use **graphical rules** (e.g., **do-calculus**) to:

- Block paths
- Identify valid adjustment sets
- Derive interventional and counterfactual distributions

This avoids ever writing messy formulas like that weird coffee equation — you reason symbolically, and then estimate quantities from data using tools like:

- Structural Equation Modeling (SEM)
- Probabilistic programming (e.g., Pyro, NumPyro)
- Causal inference libraries (like DoWhy, EconML)

Summary

You're right to be skeptical — hand-crafting expressions like:

$$Y = TU + (T - 1)(U - 1)$$

doesn't scale.

Instead, practical causal modeling moves toward:

- **Probabilistic approaches** (abduction via inference)
- **Graphical models + symbolic rules** (e.g., Pearl's do-calculus)
- **Simulation-based inference** (with neural networks or generative models)

A More Complex Example - Heart Health

Great! Let's build a **realistic SCM (Structural Causal Model)** in Python, with:

- **Multiple variables**
- **Latent (exogenous) variables and noise**
- **Causal graph structure**
- **Support for counterfactual reasoning**

We'll start simple but more realistic than the coffee example, using **graph-based reasoning + probabilistic modeling**.

Scenario: Medication → Blood Pressure → Heart Health

We'll model:

- U_1 : Genetic predisposition (latent)
- M : Medication (treatment; binary)
- BP : Blood pressure (continuous)
- HH : Heart health score (continuous)

Causal graph:

```
U1 → BP → HH
    ↑    ↑
    M    U1
```

We'll encode this as:

1. SCM structure (graph + structural functions)
2. Data generation
3. Abduction (inferring U_1 given observations)
4. Counterfactual prediction

Step 1: Install Required Libraries

We'll use `networkx` for the causal graph and `numpy/scipy` for modeling. Optional: `matplotlib` for visualization.

```
pip install networkx numpy scipy matplotlib
```

Step 2: Define the SCM in Python

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

# Fix seed for reproducibility
np.random.seed(42)

# Structural equations
def sample_U1(n):
    return np.random.normal(loc=0.0, scale=1.0, size=n)

def sample_M(n):
    return np.random.binomial(1, 0.5, size=n) # Treatment: 0 or 1
```

```

def BP_func(M, U1, noise_bp):
    return 120 - 10*M + 5*U1 + noise_bp # medication lowers BP

def HH_func(BP, U1, noise_hh):
    return 100 - 0.5*BP + 3*U1 + noise_hh # lower BP is better

# Sampling noises
def sample_noise(n, scale=1.0):
    return np.random.normal(loc=0.0, scale=scale, size=n)

# SCM sample generator
def sample_scm(n=1):
    U1 = sample_U1(n)
    M = sample_M(n)
    noise_bp = sample_noise(n, scale=2.0)
    BP = BP_func(M, U1, noise_bp)
    noise_hh = sample_noise(n, scale=1.0)
    HH = HH_func(BP, U1, noise_hh)
    return {
        "U1": U1,
        "M": M,
        "BP": BP,
        "HH": HH
    }

```

Step 3: Visualize the Causal Graph

```

def draw_causal_graph():
    G = nx.DiGraph()
    G.add_edges_from([
        ("U1", "BP"),
        ("M", "BP"),
        ("BP", "HH"),
        ("U1", "HH")
    ])
    pos = nx.spring_layout(G)
    nx.draw(G, pos, with_labels=True, node_color='lightblue', node_size=2000, font_size=16)
    plt.title("Causal Graph: Medication → Blood Pressure → Heart Health")
    plt.show()

draw_causal_graph()

```

Step 4: Abduction + Counterfactual

Let's say you observe a patient with:

- $M = 1$ (they took the medication)
- $BP = 110$
- $HH = 80$

You want to know:

What would their heart health have been if they had *not* taken the medication?

To compute this, we:

1. **Abduce:** Estimate U_1 from observed M, BP
2. **Modify:** Set $M = 0$
3. **Predict:** Recompute BP and HH

Step 5: Compute Counterfactual

```
def abduce_U1(M_obs, BP_obs):  
    # Invert BP equation to solve for U1  
    # BP = 120 - 10*M + 5*U1 + noise_bp  
    # Assume noise_bp ~ 0 for abduction  
    return (BP_obs - (120 - 10*M_obs)) / 5  
  
def counterfactual_HH(U1_est, M_cf=0):  
    noise_bp = 0 # zero for deterministic counterfactual  
    BP_cf = BP_func(M_cf, U1_est, noise_bp)  
    noise_hh = 0  
    HH_cf = HH_func(BP_cf, U1_est, noise_hh)  
    return BP_cf, HH_cf  
  
# Observed data  
M_obs = 1  
BP_obs = 110  
HH_obs = 80  
  
# Step 1: Abduce  
U1_est = abduce_U1(M_obs, BP_obs)  
print("Estimated U1 (genetic predisposition):", U1_est)  
  
# Step 2-3: Counterfactual prediction  
BP_cf, HH_cf = counterfactual_HH(U1_est, M_cf=0)  
print("Counterfactual BP if no medication:", BP_cf)  
print("Counterfactual HH if no medication:", HH_cf)
```

Example Output

Estimated U1: 1.0

Counterfactual BP: 125.0

Counterfactual HH: 73.5

Interpretation:

- This person has high sensitivity ($U1 = 1$).
- Without medication, their BP would be 125.
- Their heart health score would be worse ($HH = 73.5$ instead of 80).

Want to go further?

- Add a **confounder** (e.g., lifestyle affecting both M and HH)
- Model **stochastic counterfactuals** with Monte Carlo
- Use **Pyro** or **doWhy** to learn SCM from data and simulate interventions
- Implement **do-calculus** to compute $P(HH \mid do(M = 0))$

Let's tackle all four extensions to the heart health problem with **detailed, realistic, and executable code examples**. I'll break this down into a clear progression that builds on the prior SCM you explored (medication $\rightarrow$ BP $\rightarrow$ heart health):

4-Part Causal Inference Exploration Roadmap

Part 1: Learn an SCM from data

- We'll simulate data and then use tools like `dowhy` or `econml` to estimate causal effects.
- We'll build the graph, define assumptions, and fit causal models.

Part 2: Add confounding

- We'll introduce a confounder (e.g., lifestyle) that affects both treatment (M) and outcome (HH).
- We'll show how naive models get biased and how to adjust.

Part 3: Apply do-calculus

- We'll define the causal graph and use `dowhy`'s do-calculus-based API to compute quantities like $P(HH \mid do(M = 0))$ vs $P(HH \mid M = 0)$

Part 4: Generate probabilistic counterfactuals

- We'll perform posterior inference of exogenous variables.
- We'll simulate full **counterfactual outcome distributions** using Monte Carlo.

Part 1: Learn SCM from Data (with DoWhy)

Setup

```
pip install dowhy pandas numpy matplotlib
```

Step-by-step Code Example: Learning SCM & Estimating Effect of Medication

```
import numpy as np
import pandas as pd
import dowhy
from dowhy import CausalModel
import matplotlib.pyplot as plt

# Simulate data from the true SCM (no confounding yet)
def generate_data(n=1000, seed=42):
    np.random.seed(seed)
    U1 = np.random.normal(0, 1, size=n)           # genetic factor
    M = np.random.binomial(1, 0.5, size=n)        # medication
    noise_bp = np.random.normal(0, 2, size=n)
    BP = 120 - 10*M + 5*U1 + noise_bp
    noise_hh = np.random.normal(0, 1, size=n)
    HH = 100 - 0.5*BP + 3*U1 + noise_hh
    return pd.DataFrame({"U1": U1, "M": M, "BP": BP, "HH": HH})

df = generate_data()
df.head()
```

Build the Causal Model

```
model = CausalModel(
    data=df,
    treatment="M",
    outcome="HH",
    graph="digraph {U1 -> BP; U1 -> HH; M -> BP; BP -> HH}",
)
model.view_model()
```

This will pop up a causal graph (requires Graphviz or Jupyter integration).

Identify and Estimate Effect

```
identified_estimand = model.identify_effect()
estimate = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.linear_regression",
    control_value=0,
    treatment_value=1
)
```

```
print("Estimated Average Treatment Effect of M on HH:", estimate.value)
```

Refutation Test

```
refute = model.refute_estimate(
    identified_estimand,
    estimate,
    method_name="placebo_treatment_refuter"
)
print(refute)
```

Next: We'll add **confounding** to show how it biases estimates and how to fix it.

Part 2: Add a Confounder and Adjust

We'll now extend our simulated SCM by adding a **confounding variable**: * *L*: **Lifestyle**

- Influences **both** the likelihood of taking medication (M) **and** heart health (HH).
- For example: healthier lifestyle → less likely to need medication, better heart health.

Updated Causal Graph

```

U1 → BP → HH
  ↓   ↑   ↑
  →→ HH   ↑
L -----→ M

```

Now *L* is a **backdoor path** confounder: $M \leftarrow L \rightarrow HH$

Step-by-step Code with Confounding

```
def generate_data_with_confounder(n=1000, seed=42):
    np.random.seed(seed)
    U1 = np.random.normal(0, 1, size=n)
    L = np.random.normal(0, 1, size=n) # lifestyle confounder

    # M depends on L
    p_medication = 1 / (1 + np.exp(-1.5 * L)) # logistic function
    M = np.random.binomial(1, p_medication)

    noise_bp = np.random.normal(0, 2, size=n)
    BP = 120 - 10*M + 5*U1 + noise_bp

    noise_hh = np.random.normal(0, 1, size=n)
    HH = 100 - 0.5*BP + 3*U1 + 2*L + noise_hh # L directly affects HH
```

```

return pd.DataFrame({"U1": U1, "L": L, "M": M, "BP": BP, "HH": HH})

df_conf = generate_data_with_confounder()
df_conf.head()

```

Naive Estimate Without Adjusting for L

```

naive_model = CausalModel(
    data=df_conf,
    treatment="M",
    outcome="HH",
    graph="digraph {U1 -> BP; U1 -> HH; M -> BP; BP -> HH; L -> M; L -> HH}"
)
naive_model.view_model()

# Estimate without adjustment
estimand_naive = naive_model.identify_effect()
estimate_naive = naive_model.estimate_effect(
    estimand_naive,
    method_name="backdoor.linear_regression"
)
print("Naive estimate (without adjusting for L):", estimate_naive.value)

```

Adjusted Estimate Including L

```

adjusted_estimand = naive_model.identify_effect(proceed_when_unidentifiable=True)
estimate_adjusted = naive_model.estimate_effect(
    adjusted_estimand,
    method_name="backdoor.linear_regression",
    method_params={"backdoor_variables": ["L"]}
)
print("Adjusted estimate (with L adjusted):", estimate_adjusted.value)

```

Sanity Check: Compare Estimates

```

print("Naive ATE (confounded):", estimate_naive.value)
print("Adjusted ATE (deconfounded):", estimate_adjusted.value)

```

Interpretation

- The **naive estimate** is biased because it fails to block the backdoor path $M \leftarrow L \rightarrow HH$
- Once we **adjust for** L (by conditioning on it), we close the backdoor and get a more **causally valid estimate**

Part 3: Apply Do-Calculus with DoWhy

We'll use dowhy to: * Distinguish between **association** $P(HH \mid M = 0)$ * and **intervention** $P(HH \mid \text{do}(M = 0))$

Why this matters

Quantity	Interpretation
$P(HH \mid M = 0)$	Observed heart health among those untreated
$P(HH \mid \text{do}(M = 0))$	Expected heart health <i>if we force no one</i> to take medication

In a confounded system, these are **not the same**.

Setup: Use same confounded data and graph

```
from dowhy import CausalModel

# Use the previously generated df_conf
model_do = CausalModel(
    data=df_conf,
    treatment="M",
    outcome="HH",
    graph="digraph {U1 -> BP; U1 -> HH; M -> BP; BP -> HH; L -> M; L -> HH}"
)
model_do.view_model()
```

Identify Interventional Estimand via Do-Calculus

```
do_estimand = model_do.identify_effect()
print("Identified estimand using do-calculus:")
print(do_estimand)
```

The output might look something like:

```
Estimand type: nonparametric-ate
Backdoor criterion satisfied...
```

This tells us which variables need to be adjusted for to estimate $\text{do}(M)$ effects.

Estimate Interventional Effect

```
do_estimate = model_do.estimate_effect(
    do_estimand,
    method_name="backdoor.linear_regression"
)
print("Do-intervention estimate (ATE):", do_estimate.value)
```

Compare With Observational Association

We can also calculate $E[HH \mid M = 0]$ directly from the data:

```
print("E[HH | M=0]:", df_conf[df_conf["M"] == 0]["HH"].mean())
print("E[HH | do(M=0)]:", do_estimate.value)
```

Result Interpretation

If confounding is present: * $E[HH \mid M = 0]$ will likely be **lower** than $E[HH \mid \text{do}(M = 0)]$ * This reflects **selection bias**: people who don't take medication are systematically different

If done correctly, dowhy handles: * Identifying adjustment sets using the **backdoor criterion** * Applying the rules of **do-calculus** internally

Part 4: Generate Probabilistic Counterfactuals via Monte Carlo

Here, we go **beyond point estimates** and simulate the **full distribution** of counterfactual outcomes.

Quick Recap of What We'll Do

We simulate:

“Given someone took medication $M = 1$, had blood pressure $BP = 110$, and heart health $HH = 80$, what is the distribution of their **heart health if they had not taken the medication** ($\text{do}(M = 0)$)?”

This is true counterfactual reasoning:

1. **Abduction**: infer the latent variable U_1 using the observed world
2. **Action/Modification**: force $M := 0$
3. **Prediction**: simulate outcomes in the new (counterfactual) world

Assumptions (as before)

- SCM:
 - $BP := 120 - 10M + 5U_1 + \varepsilon_{BP}$
 - $HH := 100 - 0.5BP + 3U_1 + \varepsilon_{HH}$

We will:

- Sample noise distributions
- Invert the model to get a posterior over U_1
- Simulate outcomes under $\text{do}(M = 0)$

Step-by-step Code

```
import numpy as np
import matplotlib.pyplot as plt
```

```

# Observed values
M_obs = 1
BP_obs = 110
HH_obs = 80

# Noise assumptions
sigma_bp = 2.0
sigma_hh = 1.0

# Step 1: Abduction - sample posterior over U1 given BP_obs, M_obs
def sample_U1_posterior(bp_obs, m_obs, n=1000):
    eps_bp = np.random.normal(0, sigma_bp, size=n)
    U1_samples = (bp_obs - (120 - 10*m_obs + eps_bp)) / 5
    return U1_samples

# Step 2: Prediction under do(M=0)
def simulate_counterfactual_HH(U1_samples, m_cf=0):
    eps_bp_cf = np.random.normal(0, sigma_bp, size=len(U1_samples))
    BP_cf = 120 - 10*m_cf + 5*U1_samples + eps_bp_cf

    eps_hh_cf = np.random.normal(0, sigma_hh, size=len(U1_samples))
    HH_cf = 100 - 0.5*BP_cf + 3*U1_samples + eps_hh_cf
    return HH_cf

# Run it
U1_posterior = sample_U1_posterior(BP_obs, M_obs)
HH_cf_samples = simulate_counterfactual_HH(U1_posterior)

# Plot result
plt.hist(HH_cf_samples, bins=40, alpha=0.7, label="Counterfactual HH (do(M=0))")
plt.axvline(x=HH_obs, color='r', linestyle='--', label="Observed HH")
plt.title("Counterfactual Heart Health Distribution")
plt.xlabel("Heart Health")
plt.ylabel("Frequency")
plt.legend()
plt.grid(True)
plt.show()

```

Interpretation

- The histogram shows the **distribution of what your heart health would have been** if you hadn't taken medication.
- The red line shows your **actual observed** outcome.

You now have a full **probabilistic view**:

- Mean, variance, quantiles
- Confidence that the medication helped or not
- Uncertainty modeled through noise + posterior samples

Mediation Analysis

Instrumental Variables

Time series SCMs